Secure IoT Device Authentication Using Lightweight Cryptography

(SIOTDA)

Information Security

by

N Ravi Tejesh
(Roll No: CS23B2051)

N Durgalakshmi Prasad
(Roll No: CS23B2052)

Under the guidance of

Dr. Bhale Pradeepkumar Gajendra

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, DESIGN AND MANUFACTURING, KANCHEEPURAM

April 22, 2026

List of Figures

14.1 Device Registration and Master Device Selection 34
14.2 Secure Chat Interface from Device 1 35
14.3 Secure Chat Interface from Device 2 35
14.4 Attack Simulation Interface 36
14.5 Cryptography Simulation and Avalanche Effect 37
14.6 Authentication Flow Simulation 38

Contents

1 Introduction **5**
1.1 Background of IoT Security . . . 5
1.2 Why Device Authentication is Critical . . . 5
1.3 Problems in Existing IoT Authentication Systems . . . 5

2 Problem Statement **6**
2.1 Replay Attacks . . . 6
2.2 Device Spoofing . . . 6
2.3 Man-in-the-Middle (MITM) Attacks . . . 7
2.4 Weak IoT Authentication . . . 7

3 Objectives **8**

4 System Architecture **9**
4.1 Three-Tier Architecture Description . . . 9
4.2 Frontend Layer Structure . . . 9
4.3 Backend Layer . . . 9
4.4 WebSocket Layer . . . 10
4.5 Database Layer . . . 10

5 Tecnologies Used **11**
5.1 Python 3 and Flask . . . 11
5.2 Flask-SocketIO and Eventlet . . . 11
5.3 SQLite . . . 11
5.4 JavaScript and Web Crypto API . . . 12
5.5 AES-256-GCM . . . 12
5.5.1 AES-256-GCM Representation . . . 12
5.6 HMAC-SHA256 . . . 12

6 Detailed Explanation of the Module **13**
6.1 Backend Modules . . . 13
6.1.1 app.py - Application and WebSocket Setup . . . 13
6.1.2 routes.py — REST API Layer . . . 13
6.1.3 `database.py` - Database Operations . . . 14
6.1.4 *crypto.py — Cryptographic Operations* . . . 14
6.2 Frontend Modules . . . 14
6.2.1 script.js — The Main Functionality of the Frontend . . . 14
6.2.2 Client-Side Encryption . . . 15
6.2.3 WebSocket Connection . . . 15

6.2.4 Chat Module . . . 15
6.2.5 Dashboard of Security . . . 16
6.2.6 Attack Simulation Module . . . 16
6.2.7 Authentication Flow Visualization . . . 16

7 API Design **17**
7.1 GET /api/stats . . . 17
7.2 POST /api/register . . . 17
7.3 GET /api/devices . . . 18
7.4 POST /api/auth/flow . . . 18
7.5 POST /api/attack . . . 18
7.6 GET /api/logs . . . 19
7.7 GET /api/messages . . . 19

8 Algorithms Used **20**
8.1 HMAC-SHA256 . . . 20
8.1.1 Mathematical Foundation . . . 20
8.1.2 Message Construction . . . 20
8.1.3 Security Properties . . . 20
8.2 Prevention Against Replay Attacks Using Nonce . . . 21
8.2.1 Creation of Nonce . . . 21
8.2.2 Lifecycle State Machine . . . 21
8.2.3 Replay Prevention Logic . . . 21
8.3 AES-256-GCM Authenticated Encryption . . . 22
8.3.1 Construction Overview . . . 22
8.3.2 Key Parameters . . . 22
8.3.3 Storage Format . . . 22
8.3.4 IV Uniqueness Guarantee . . . 22

9 Attack Simulation **23**
9.1 Replay Attack . . . 23
9.1.1 Attack Scenario . . . 23
9.1.2 Why the Attack Fails . . . 23
9.2 Device Spoofing Attack . . . 23
9.2.1 Attack Scenario . . . 23
9.2.2 Why This Attack Fails . . . 24
9.3 Device Spoofing Attack . . . 24
9.3.1 Attack Scenario . . . 24
9.3.2 Why This Attack Does Not Work . . . 24

10 Real-Time Communication **25**
10.1 WebSocket and HTTP Polling . . . 25
10.2 Room Architecture for Socket.IO . . . 25
10.3 Real-time Dashboard Update . . . 26
10.4 Connection Status Handling . . . 26

11 Database Design **26**
11.1 Schema Description . . . 26
11.2 Table: devices . . . 27
11.3 Table: nonces . . . 27
11.4 Table: sessions . . . 28
11.5 Table: messages . . . 28
11.6 Table: logs . . . 28
11.7 Other Observations . . . 29

12 System Workflow **29**
12.1 Device Registration . . . 29
12.1.1 Authentication Process . . . 29
12.1.2 Secure Messaging . . . 30
12.2 Attack Detection . . . 30
12.3 Dashboard Real-Time Update . . . 31

13 Running Procedure **31**
13.1 Prerequisites . . . 31
13.1.1 Step 3: Project Directory Structure . . . 32
13.1.2 Step 4: Start the Backend Server . . . 32
13.1.3 Step 5: Open the Application . . . 32
13.1.4 Step 6: Register a Device . . . 32
13.1.5 Step 7: Register a Second Device and Start Chat . . . 33
13.1.6 Step 8: Run Attack Simulations . . . 33

14 Results and Output **34**
14.1 Device Registration and Master Selection . . . 34
14.2 Chat Section (Device 1) . . . 35
14.3 Chat Section (Device 2) . . . 35
14.4 Attack Simulation . . . 36
14.5 Cryptography Simulation and Avalanche Effect . . . 37
14.6 Authentication Flow Simulation . . . 38
14.7 Backend Execution and Terminal Output . . . 38
14.8 Limitations . . . 39

15 Conclusion **40**

16 References **40**

Chapter 1

Introduction

There have been many advancements regarding the interconnectivity of various gadgets. There has been an increase in the number of gadgets that were earlier not connected but now interact in real-time to perform some specific tasks. This has led to the automation of many processes and made their functionality better. But it also brings with it some problems related to security.

An important problem related to the security of the Internet of Things gadgets is that most of these gadgets do not have much processing power and memory capacity because of which security features cannot be added successfully. As a result, these devices become an easy target for cyber-attacks.

In this project, we will try to design such a solution for devices to check the legitimacy of the other device before proceeding with any kind of communication.

1.1 Background of IoT Security

The increase in internet connectivity of devices has led to the emergence of major security threats. Most of the Internet-of-Things applications developed in the real world are configured improperly using default protocols or ineffective security procedures.

IoT systems differ from conventional computer systems in that they have distributed architectures comprising diverse device types. Maintaining security in such systems is not easy. Security issues are ignored in favor of lower costs and improved efficiency.

1.2 Why Device Authentication is Critical

Prior to communication between devices, it is necessary to ensure that the devices are genuine. In case of poor authentication, unauthorized devices may join the network and create issues such as transmitting erroneous information or disrupting the process.

Good authentication guarantees that only reliable devices can transmit messages.

1.3 Problems in Existing IoT Authentication Systems

Various authentication schemes currently in use make use of heavy cryptography, such as RSA and TLS. Even though they are reliable, these techniques are unsuitable for IoT due

to their heavy computation burden.

As a result, there have been other schemes that can easily be hacked. The following are among the most common problems encountered:

- Weak or static credentials used
- No protection from replay attacks
- Unsecured transmission channels
- Scaling up devices

To tackle these problems, this work implements a simple authentication scheme using a challenge-response protocol involving nonces.

Chapter 2

Problem Statement

The fast development of IoT systems has created various security issues, particularly in device authentication. The devices are designed to function with constrained capabilities and cannot accommodate sophisticated security schemes.

A trade-off can be found in current security solutions. The robustness of such schemes can be compromised due to their high computation cost. On the other hand, lightweight security schemes cannot provide adequate protection against intrusions.

The goal of this project is to develop an efficient yet robust scheme that can be used in IoT systems.

2.1 Replay Attacks

Replay attacks occur where the adversary intercepts the legitimate data packet and repeats its transmission at a later time to gain access. Without verification methods such as nonce verification and timestamp, these types of attacks can be executed successfully.

These kinds of attacks can be performed with ease in the absence of these measures.

2.2 Device Spoofing

In device spoofing, the attacker acts like an authorized device by presenting authentic credentials in the form of device ID. If the system uses such credentials for device verification, then it is very easy to spoof the device.

Such attacks may result in erroneous data insertion and operational disruption.

2.3 Man-in-the-Middle (MITM) Attacks

The attack happens when the perpetrator is positioned between two communicating devices, thus making it possible for him to either monitor or tamper with the messages without being noticed.

When the integrity of the message cannot be validated, this sort of attack becomes extremely dangerous.

2.4 Weak IoT Authentication

Because of hardware constraints, most IoT devices rely on simple authentication schemes. This makes the system fast but lowers its security level.

Problems encountered are:

- Inadequate credentials
- Vulnerability to typical attacks
- Message integrity checks not performed

Therefore, there is a need for a more effective method.

Chapter 3

Objectives

This project aims to develop a lightweight authentication framework for the IoT using cryptography. The main purpose is to allow access to the IoT network only by legitimate devices and prevent others from using it.

The following objectives have been set up for this project:

- To design a lightweight authentication mechanism that runs efficiently on IoT devices that are resource-constrained.
- To implement the challenge-response system with the help of nonces to ensure that each response cannot be used more than once.
- To defend against replay attacks, device impersonation, and MITM attacks using validation and proper cryptographic algorithms.
- To authenticate IoT devices with HMAC-SHA256 without sending any critical information like PSKs through the channel.
- To allow communication between devices securely using AES-256-GCM to provide both confidentiality and integrity.
- To construct a functional system that incorporates the frontend, backend, and database systems in addition to the authentication, encryption, and attack simulation features.
- To create an interactive UI that illustrates authentication, encryption, and attack simulation.
- To achieve scalability of this system without compromising its performance and security.

In summary, this research will develop a feasible framework that provides effective authentication and data protection solutions for the IoT devices.

Chapter 4

System Architecture

4.1 Three-Tier Architecture Description

This system architecture consists of three layers: frontend, backend, and database. Every layer is responsible for some specific functionality to make the software structure simpler.

The frontend serves as the user interface; it enables interactions with a system. Meanwhile, the backend carries out the core functionalities of the software along with all the security procedures. A database layer contains all the data needed by the program. In addition to those, a WebSocket layer is utilized to provide live connection between frontend and backend components.

Since this system works locally, its structure can remain light and independent of any external components or systems.

4.2 Frontend Layer Structure

The frontend of this application is implemented in HTML, CSS, and JavaScript. It includes several pages such as registration of devices, chatting, system dashboard, visualization of the authentication flow, and attacks simulation.

There are two major types of communication between frontend and backend:

- **REST API:** Used for registration of devices, initiating authentication procedure, and requesting data
- **WebSocket:** Required for providing users with live information such as messaging and dashboard updates

In terms of security measures, cryptographic functions are executed in-browser using the Web Crypto API.

4.3 Backend Layer

Flask framework is utilized in backend implementation where multiple modules have been implemented to increase ease of use:

- **app.py**: Application setup and management of websockets
- **routes.py**: REST API endpoints implementation

- **crypto.py**: Cryptography procedures
- **database.py**: Database procedures management

Asynchronous operation is provided with use of Flask-SocketIO module and eventlet package, providing the ability to manage several simultaneous requests.

4.4 WebSocket Layer

WebSocket protocol is employed to provide constant communication. Asynchronous communication reduces need to establish connections with every request by setting one constant connection.

Each device is connected to its particular room which allows sending messages to a required recipient.

Main operations performed in this layer are:

- connect, disconnect
- join
- send_message, receive_message
- devices_update, log_update

These changes lead to increased system performance.

4.5 Database Layer

SQLite database is chosen due to simplicity and low overhead. All the data in SQLite database is stored in a single file.

Write-ahead logging is employed to increase speed of writing data to the database by allowing simultaneous write and reading.

Tables which contain essential data in this database are:

- devices
- nonces
- sessions
- messages
- logs

These tables contain data which include device data, authentication data, message data, and logging data. This kind of organization makes the system orderly and everything is well documented.

Chapter 5

Tecnologies Used

5.1 Python 3 and Flask

Backend development in the present project is based on Python 3 programming language with the use of the Flask framework. Flask is a web framework with a minimum of functions required for routing requests, managing sessions, and generating JSON replies. Due to its simple structure, Flask enables efficient development of an API without increasing system complexity.

For organization of REST API endpoints, blueprints are implemented in *routes.py*, while their linking in *app.py*. Flask works effectively with Python built-in sqlite3 module, allowing to perform all the necessary actions without usage of ORM (such as SQLAlchemy).

5.2 Flask-SocketIO and Eventlet

To implement real-time data exchange between the server and the client, Flask-SocketIO is used to enable communication via WebSockets without constantly making HTTP requests.

Among the advantages of Socket.IO is automatic reconnection feature and availability of room functionality, which allows limiting delivery of data only to the intended recipient. In this application, each device is assigned to a certain room.

In addition, Eventlet is utilized in order to implement asynchronous functions in the backend. Usage of lightweight threads allows handling multiple connections without any negative effect on performance.

5.3 SQLite

In this project, SQLite is used as the underlying database engine. Being a lightweight and compact solution, it allows one to store all the information in a single file, thereby ensuring simplicity of configuration and use.

All manipulations with the database are done using Python's native *sqlite3* library, thus avoiding any additional requirements and ensuring that no external dependencies are needed for the program.

Performance can be increased by enabling WAL mode, which enables the efficient processing of read/write requests, especially during simultaneous actions, typical for real-time environments.

5.4 JavaScript and Web Crypto API

The development of the front end was done in vanilla JavaScript. As far as security-related functionality is concerned, the Web Crypto API was used natively in the browser, allowing for native implementations of cryptographic routines such as hashing and encryption.

The main client-side operations include:

- Generating SHA-256 hashes for key derivation
- Importing keys for encryption and decryption
- Encrypting and decrypting messages using AES-256-GCM
- Generating secure random values for initialization vectors (IVs)

This approach ensures that cryptographic processing is handled securely without relying on external libraries.

5.5 AES-256-GCM

AES-256-GCM is used to provide secure message encryption, ensuring both confidentiality and data integrity.

Each message is encrypted using a 256-bit key along with a randomly generated initialization vector (IV), producing ciphertext along with an authentication tag. Any modification to the encrypted data causes the decryption process to fail.

In this system, the encryption key is derived using SHA-256, and a new random IV is generated for every message to maintain security.

5.5.1 AES-256-GCM Representation

AES-256-GCM performs authenticated encryption as:

$$C,\ T = \text{AES-GCM}_K(IV,\ P)$$

5.6 HMAC-SHA256

HMAC-SHA256 is used for device authentication by generating a message authentication code using a shared PSK.

The calculation of the HMAC using the challenge is performed on the device, while verification is done on the server side through the same calculation. If the two calculations match, then the device is verified. This algorithm makes sure that the PSK is not communicated through the network.

Chapter 6

Detailed Explanation of the Module

6.1 Backend Modules

6.1.1 app.py - Application and WebSocket Setup

The `app.py` file acts as the primary point of execution for our backend server code. It coordinates various components within the system.

In this module, we initiate the Flask app and set it up to be used throughout the project. We also enable CORS, configure Socket.IO, and include all routes in a single place where our application runs flawlessly.

In addition, `app.py` consists of auxiliary functions that propagate notifications of device lists and system logs, among other information. As a result, users do not need to refresh pages in order to see any change from their backend server.

Moreover, we handle several WebSocket events in `app.py`, namely establishing connection with the client, sending and receiving messages between devices, and leaving rooms.

6.1.2 routes.py — REST API Layer

In the `routes.py` file, you will find all the RESTful APIs that serve as endpoints for the frontend. The endpoints are structured using Flask Blueprints in order to make sure that the code remains modular and scalable.

These APIs perform the following operations:

- Registering devices
- Obtaining the device list
- Performing the simulation of the authentication process
- Simulation of attacks
- Extracting system logs
- Obtaining some statistics about the system

All endpoints receive the data from the front-end, process it accordingly, connect with the database, if necessary, and produce a response in JSON format.

For example, the authentication API demonstrates the whole challenge-response scheme in action. Likewise, the attack API can be used to test how the system performs under different kinds of attacks.

6.1.3 `database.py` - Database Operations

The `database.py` file is responsible for all the database management operations in the system, which are done using the SQLite database.

It performs the following functions:

- Creates the necessary database tables on initialization of the system
- Database connection management
- Querying the database for storing and retrieving data

Information such as the device information, nonces used for authentication, sessions, encrypted data, and system logs are kept in the database. This ensures that all the activity in the system can be tracked and accessed whenever needed.

6.1.4 *crypto.py — Cryptographic Operations*

The *crypto.py* file is responsible for handling all the necessary cryptographic operations within the system. This process ensures both the security of the authentication and the communications.

The primary functions that take place in the file are the following:

- Creation of HMAC hashes for the purpose of authenticating
- Validation of HMAC hashes sent by the device
- Random generation of nonce values
- Session token creation
- Encryption and decryption using AES-256-GCM

With such a set of operations, devices can be authenticated properly and data is not leaked during the communication.

6.2 Frontend Modules

6.2.1 script.js — The Main Functionality of the Frontend

The `script.js` file holds the basic functionality of the frontend. It deals with the interaction between users, sends API requests to the backend, and updates the frontend according to the information returned from the backend.

The major functions of this file include:

- Making requests to the backend using API calls
- Maintaining WebSocket communication
- Initializing the page and dealing with user events
- Updating and displaying data in the interface

This file connects the user interface with the backend in order to ensure smooth application performance.

6.2.2 Client-Side Encryption

Here, all the processes of encrypting and decrypting occur directly at the frontend via the use of Web Crypto API of the browser.

It includes:

- Generation of the encryption keys based on shared secrets
- Message encryption before the message transmission
- Message decryption after receiving the message

This way of operating allows keeping data safe already at the client's computer before the message transmission to the server and providing the user with additional safety.

6.2.3 WebSocket Connection

The frontend establishes the WebSocket connection to the backend via Socket.IO. The connection remains open allowing to communicate continuously between client and server.

WebSocket protocol allows implementing:

- Instantaneous message exchange
- Instant updating of the list of devices
- Updates of the information on the dashboard instantly
- Event-based communication

With the WebSocket connection, there is no need to make any new requests constantly.

6.2.4 Chat Module

This module is designed for enabling secure messaging between devices. The user will be able to communicate via messaging while making sure that the data stays safe.

Key functionalities of this module include:

- Device selection
- Message exchange
- Encryption of messages sent out
- Decryption of incoming messages

Moreover, there is an ability to simulate a man-in-the-middle attack within this module.

6.2.5 Dashboard of Security

The security dashboard will provide an overview of the current situation in the system. It will give insights into ongoing processes within the system.

Among other pieces of data, it will present:

- List of registered devices
- Authentication results
- Attack detection notifications
- System logs

Moreover, it will display the latest information about the current state of the system.

6.2.6 Attack Simulation Module

This module provides users with the ability to conduct simulations of various attacks, including replay attacks, spoofing, and man-in-the-middle attacks. This module serves mainly to demonstrate how these attacks function in real-world scenarios.

Users are able to use this module to:

- Learn how different attacks are performed
- Study the detection and prevention process for each attack
- Evaluate the effectiveness of the security mechanisms implemented

Ultimately, this will give users a better understanding of how the system responds to each type of attack.

6.2.7 Authentication Flow Visualization

This module visualizes the process of authentication from start to finish. This makes it easier to understand each step of the authentication process, from requesting device access to creating a session.

The authentication flow visualization includes steps like:

- Requesting device access
- Generating a nonce
- Calculating the HMAC value
- Authenticating the answer
- Creating a session

Chapter 7

API Design

The backend provides RESTful API endpoints responsible for handling communication between the frontend and the server. The design of these APIs is straightforward and adheres to the JSON structure, ensuring simplicity and ease of testing.

Every API is dedicated to a particular purpose, including device registration, authentication, fetching data, or attack simulation. Basic HTTP methods such as GET and POST are employed consistently across the application.

7.1 GET /api/stats

This API endpoint retrieves the necessary system statistics displayed in the dashboard. It yields consolidated results like the count of registered devices, the total number of messages transferred, and the count of identified security threats.

Function: System statistics retrieval for the dashboard.

Response:

```
{
  "devices": 5,
  "messages": 120,
  "attacks_detected": 3
}
```

7.2 POST /api/register

This is an endpoint through which one can register a device with the system by providing a unique device ID and secret key.

Purpose: Registration of a new device

Request:

```
{
  "deviceId": "device_1",
  "secretKey": "my_secret_key"
}
```

Response:

```
{
  "success": true,
  "device_id": "device_1"
```

```
}
```

7.3 GET /api/devices

This endpoint provides a listing of all the devices registered with the system and can be used on the front end to display devices that can be selected by the user.

Purpose: Retrieval of all registered devices.

Response:

```
{
  "devices": ["device_1", "device_2"]
}
```

7.4 POST /api/auth/flow

This API endpoint is intended for testing the entire process of authentication using the challenge-response paradigm. It creates a nonce, calculates an HMAC, and then checks the validity of the device.

The endpoint also offers different ways of operating depending on how we need to test the system:

- normal
- wrong_psk
- replay

Purpose: For demonstrating the authentication flow.

Request:

```
{
  "device_id": "device_1",
  "mode": "normal"
}
```

Response (simplified):

```
{
    "status": "success",
    "nonce": "abc123",
    "hmac": "xyz456"
}
```

7.5 POST /api/attack

This API endpoint is responsible for simulating various kinds of attacks like replay, spoofing, and man-in-the-middle attack. Depending on the chosen attack method, the system will perform the attack simulation and provide the results.

Purpose: To demonstrate how the system reacts to various attack types.

Request:

```
{
  "type": "replay",
  "device": "device_1"
}
```

Response:

```
{
  "result": "Replay attack detected and blocked"
}
```

7.6 GET /api/logs

This endpoint is used to fetch the system logs, which include authentication processes and attacks that have been detected by the system. The log files are primarily used for the purpose of viewing system activities and alerts on the dashboard.

Purpose: To view system activities and security events.

Response:

```
[
 {
 "event": "AUTH",
    "device": "device_1",
    "description": "Authentication successful"
  }
]
```

7.7 GET /api/messages

This endpoint returns stored messages exchanged between devices. The messages are encrypted and include metadata such as sender, receiver, and timestamp.

Purpose: Retrieve communication history.

Response:

```
[
  {
    "sender": "device_1",
    "receiver": "device_2",
    "ciphertext": "encrypted_data",
    "timestamp": "2026-04-22"
  }
]
```

Chapter 8

Algorithms Used

This chapter describes the main algorithms used in the system for authentication, replay protection, and secure communication. The focus is on using cryptographic techniques that are both secure and efficient enough for IoT environments.

8.1 HMAC-SHA256

HMAC-SHA256 is used for device authentication. It ensures that only devices with the correct PSK are able to generate a valid response.

8.1.1 Mathematical Foundation

HMAC (Hash-based Message Authentication Code) is defined in RFC 2104. Given a hash function H:

$$HMAC(K, M) = H\big((K_0 \oplus opad) \parallel H\big((K_0 \oplus ipad) \parallel M\big)\big)$$

For SHA-256:

- Block size $B = 64$ bytes
- Output size $L = 32$ bytes

8.1.2 Message Construction

In this system, the message is constructed as:

$$M = device_id \parallel timestamp \parallel nonce \parallel payload$$

The same message format is used on both the backend and frontend, ensuring consistent HMAC computation.

8.1.3 Security Properties

- **Authentication:** Confirms the identity of the device using a shared PSK
- **Integrity:** Ensures that the message has not been altered
- **Key Protection:** The PSK is never transmitted over the network

8.2 Prevention Against Replay Attacks Using Nonce

Replay attacks are avoided through use of a unique nonce with every attempt at authentication. The fact that each request gets a different value ensures that no previously intercepted message can be used again.

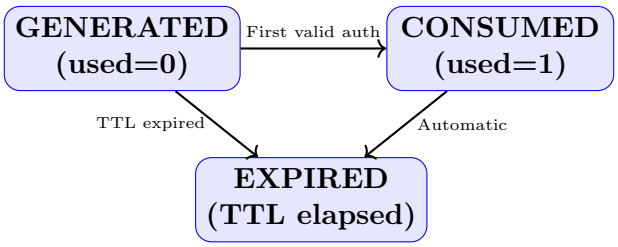
8.2.1 Creation of Nonce

A nonce can be generated from a secure random source as follows:

$$\text{nonce} = \text{SecureRandom}(128\text{-bit})$$

8.2.2 Lifecycle State Machine

Each nonce follows a simple lifecycle:



- Generated
- Issued to the device
- Used during authentication
- Expired after use or timeout

8.2.3 Replay Prevention Logic

Before accepting a request, the system verifies:

- Whether the nonce exists
- Whether it has already been used
- Whether it has expired

If any of these checks fail, the request is rejected.

Listing 8.1: Nonce validation logic

```
def is_nonce_valid(nonce):
    row = get_nonce(nonce)

    if not row:
        return False
    if row.used:
        return False
```

```
    if current_time() > row.expiry:
        return False

    return True
```

8.3 AES-256-GCM Authenticated Encryption

AES-256-GCM is used to secure communication between devices. It provides both encryption and integrity protection.

8.3.1 Construction Overview

AES-256-GCM combines encryption and authentication:

$$C, T = \text{AES-GCM}_K(IV, P)$$

where:

- P = plaintext
- C = ciphertext
- T = authentication tag

8.3.2 Key Parameters

Parameter	Value	Description
Key size	256 bits	Encryption key
IV size	96 bits	Random per message
Tag size	128 bits	Integrity check

8.3.3 Storage Format

Encrypted messages are stored in the following format:

$$IV : Ciphertext : Tag$$

This ensures that all required components are available for decryption.

8.3.4 IV Uniqueness Guarantee

A new random IV is generated for each encryption operation. This ensures that even if the same message is sent multiple times, the resulting ciphertext will be different, preventing pattern-based attacks.

Chapter 9

Attack Simulation

This chapter demonstrates how common network attacks are simulated within the system and explains how the proposed authentication mechanism handles and prevents them.

9.1 Replay Attack

9.1.1 Attack Scenario

A replay attack occurs when an attacker captures a valid authentication message and attempts to reuse it later to gain unauthorized access.

The sequence of actions is as follows:

- The server generates and sends a nonce to the device.
- The device computes an HMAC using the nonce and returns it.
- The attacker intercepts this valid request.
- At a later time, the attacker resends the same request to the server.

9.1.2 Why the Attack Fails

The system prevents replay attacks using strict nonce validation.

- Each nonce is marked as **used** after a successful authentication.
- If the same nonce appears again, the request is immediately rejected.
- Nonces also have a limited lifetime (TTL), after which they become invalid.

Because of these checks, even a valid captured message cannot be reused. The system identifies that the nonce is no longer valid and blocks the request.

9.2 Device Spoofing Attack

9.2.1 Attack Scenario

In a device spoofing attack, an attacker attempts to impersonate a legitimate device by using its identifier.

- The attacker obtains or guesses a valid device ID.
- An authentication request is sent using that identity.
- The server responds with a nonce as part of the challenge.
- The attacker tries to generate a valid response without the PSK.

9.2.2 Why This Attack Fails

In order to counter replay attacks, strict checks are done on the nonces here.

- When a nonce is successfully authenticated, it is flagged as **used**.
- Should a nonce be repeated, it will result in rejection of the request.
- Nonces can only be valid for a certain period of time (TTL) after which they expire.

This means that even if a message gets captured, it cannot be used for any authentication purposes since the system detects that it is no longer a valid nonce.

9.3 Device Spoofing Attack

9.3.1 Attack Scenario

Here, the idea behind the attack is to make the server think that you are a device by sending your identifier.

- The attacker either gets or assumes the device ID.
- Authentication request is sent with device identity.
- Server sends back a nonce as part of challenge.
- Now attacker needs to generate a correct response using a device identity but without PSK.

9.3.2 Why This Attack Does Not Work

The system employs AES-256-GCM cryptography to protect data confidentiality and integrity.

- Each message has an authentication tag.
- Any alteration to the message results in a failed tag check.
- These altered messages are immediately discarded.

As such, any modifications and injection attacks cannot be performed undetected, and the communication process between the devices is safe.

Chapter 10

Real-Time Communication

10.1 WebSocket and HTTP Polling

In many conventional web-based applications, updates are retrieved through periodic requests made to the server (say, calling `/api/logs` periodically). Although such an implementation is rather easy, it is far from optimal in terms of efficiency for systems requiring real-time updates.

Among others, some of the downsides are as follows:

- **Latency:** In case any event occurs right after an update request, it would be too late for the user to get this update until the next iteration.
- **Overhead traffic:** Requests are made continuously irrespective of the availability of the actual data, thus wasting bandwidth resources.
- **Extra server load:** Constant polling activities cause additional server load especially in case of a large number of users.

To address these problems, WebSocket technology is employed, which opens a persistent channel after a brief initial connection process. After establishing this connection, data can be transferred from the server side to clients immediately upon its availability.

10.2 Room Architecture for Socket.IO

The Flask-SocketIO library provides a communication approach based on rooms, which means that messages can be delivered only to particular clients rather than all clients simultaneously.

Upon selecting a device, the front-end triggers the `join` event. Then, the server will place the client in a room using `join_room(device_id)`. Each device will have its room through which it will communicate.

For instance, when sending a message from one device to another, the server will send the message only to the appropriate rooms:

```
socketio.emit("receive_message", msg, room="Device_B")
socketio.emit("receive_message", msg, room="Device_A")
```

As a result, only the specified devices will get the message while others will remain unaffected.

10.3 Real-time Dashboard Update

Updates to the dashboard occur in real-time through listening to the `log_update` event via the WebSocket:

```
sock.on("log_update", () => {
  if (document.getElementById("logTableBody")) loadDashboard();
});
```

Whenever there is any notable event in the system, such as successful authentication, detection of an attack, or registering a new device, the backend triggers a `log_update` event for all clients connected to it.

Once the frontend client gets this signal, it calls the REST API to get the latest data and updates the dashboard almost instantaneously. This provides a mechanism to keep the information displayed updated without requiring refreshing manually.

10.4 Connection Status Handling

The `connection-bar` displayed at the top of every page represents the status of the WebSocket connection. Initially, it shows "Connecting...". After the connection establishment, it updates to "Connected" in green color. In case of failure in the connection, the bar turns into "Offline" mode in red color.

In the offline state, the user cannot send any message or execute simulation commands.

Chapter 11

Database Design

This chapter describes how data is structured in the database and where it is stored. The SQLite database is chosen because of its lightness, file-based structure, and simple usage. No additional database server is necessary, and all the data is saved in one database file called `iot_security.db`.

The initialization process of the database takes place during startup through the function `init_db()`. The function checks whether there are necessary tables in the database and creates them if it is not true.

11.1 Schema Description

The database consists of five tables:

- devices

- nonces
- sessions
- messages
- logs

Every table serves its specific function, e.g., saving device data, authentication, communication, logging, etc.

11.2 Table: devices

The `devices` table contains information about all IoT devices that have been registered in the system.

- **device_id:** Unique identifier for the device (Primary Key)
- **psk:** Pre-shared key for the authentication process
- **registered_at:** Timestamp marking the registration date of the device

This table is primarily accessed during the authentication process to check the authenticity of the device and get its PSK.

11.3 Table: nonces

The `nonces` table maintains a record of the challenge values involved in the authentication process.

- **nonce:** Unique random number (Primary Key)
- **device_id:** Device ID linked with the nonce
- **created_at:** Timestamp of creation of the nonce
- **used:** Boolean value specifying whether the nonce has been used (0 or 1)
- **ttl:** Time-to-live value specifying the duration of the nonce validity

The `device_id` column has a foreign key relationship with the `devices` table. This table is crucial in defending against any replay attack.

11.4 Table: sessions

The `sessions` table contains the sessions tokens created after a successful authentication of a particular device.

- **token:** Unique session token (Primary Key)
- **device_id:** Device associated with the particular session
- **created_at:** Timestamp for the creation of session
- **expires_at:** Timestamp for the expiry of session

The use of this table ensures that sessions are managed correctly and that the expired sessions are not reused.

11.5 Table: messages

Messages are stored in this `messages` table, which includes encrypted messages.

- **id:** Message id (Primary key)
- **sender:** Device id of the sender
- **receiver:** Device id of the receiver
- **ciphertext:** Encrypted message content
- **iv:** Initialization vector for encryption of message
- **timestamp:** Message sending timestamp

All messages are stored in the encrypted format hence ensuring confidentiality.

11.6 Table: logs

This `logs` table is designed for logging all system events and security incidents.

- **id:** Autonumbered log ID (primary key)
- **timestamp:** The time at which the event took place
- **event:** Event type (such as AUTH, REPLAY, MITM)
- **device:** Device associated with the event
- **description:** Event details

These help monitor system activity, debug problems, and analyze how the system reacts to various security events.

11.7 Other Observations

The database operates in Write-Ahead Logging (WAL) mode, which lets read operations occur simultaneously with write operations. This is helpful during real-time usage when the data is being updated often.

All database-related activities are carried out using the Python's `sqlite3` library. This library works on row-level access that makes it easy to access data by their name rather than by position.

Chapter 12

System Workflow

In this chapter, I will provide brief insight into the work of our system, beginning from device registration to authentication. I will present you with interaction between various components while the system operates and security is being taken care of.

12.1 Device Registration

1. User visits the `register.html` page.
2. Device ID and Pre-Shared Key are supplied.
3. Data validation is performed on the frontend side.
4. Post request is made to `POST /api/register`.
5. Validity is checked, and duplicate devices are filtered out by the backend.
6. Device information is saved to the `devices` table.
7. Event `REGISTER` is recorded in the logs.
8. Event `devices_update` is broadcasted to all users.
9. Device lists are automatically updated everywhere.
10. Device becomes the master device.

12.1.1 Authentication Process

1. The user picks a device and initiates the authentication process.
2. The frontend makes a call to `POST /api/auth/flow`.

3. The backend fetches the PSK from the database.
4. A nonce is generated randomly.
5. The HMAC is generated based on device information and nonce.
6. If all goes well, the authentication is successful.
7. If not, it fails (wrong key, replay attack, etc.).
8. The frontend shows the authentication process.
9. A `log_update` event is sent to all clients.

12.1.2 Secure Messaging

1. Devices sending and receiving the message should be registered.
2. User launches the `chat.html` page and chooses the destination device.
3. Computes a symmetric AES-256 key based on SHA-256.
4. Generates a random IV.
5. Encrypts the message with AES-256-GCM algorithm.
6. Sends the encrypted message via WebSocket channel.
7. Saves the message into the database in the encrypted form.
8. Delivers the message to the receiver.
9. The receiver will decrypt the message.

12.2 Attack Detection

1. The user navigates to the `attack.html` page.
2. A device and an attack type are chosen.
3. The frontend initiates a POST request to `/api/attack`.
4. The backend simulates an attack using the specified method.
5. In case of detection, the attack is blocked and logged.
6. A `log_update` message is dispatched to all connected clients.
7. The frontend displays the outcome in a console-like format.
8. In the case of an active dashboard, it updates itself.

12.3 Dashboard Real-Time Update

1. The user opens `dashboard.html`.
2. Initial data is loaded from `/api/stats` and `/api/logs`.
3. The data is displayed using charts and tables.
4. A WebSocket listener is activated for live updates.
5. Whenever an event occurs (authentication, attack, or registration), the server emits `log_update`.
6. The dashboard receives this event and refreshes automatically.
7. Updated information is displayed instantly without requiring a page reload.

Chapter 13

Running Procedure

This chapter describes the steps required to set up and run the project.

13.1 Prerequisites

Requirements:

- Python 3.9 or later
- Required libraries: flask, flask-socketio, flask-cors, eventlet, cryptography
- A web browser (Chrome, Firefox, Edge, or Safari)

Installation Steps:

- Clone or download the project repository from GitHub:
  - **GitHub Repository Link**
- Navigate to the backend directory:
  - **cd backend**
- Install the required dependencies:
  - **pip install flask flask-socketio flask-cors eventlet cryptography**

The main dependencies include Flask for backend development, Socket.IO for real-time communication, Eventlet for asynchronous processing, and Cryptography for secure operations.

13.1.1 Step 3: Project Directory Structure

Ensure that the project is organized as follows:

```
siotda/
  backend/
    app.py
    routes.py
    database.py
    utils/
      __init__.py
      crypto.py
  frontend/
    index.html
    register.html
    chat.html
    dashboard.html
    attack.html
    flow.html
    crypto.html
    encryption.html
    script.js
    styles.css
```

13.1.2 Step 4: Start the Backend Server

- python app.py Frontend

After running the above command, the server will start at:

- http://localhost:5000

13.1.3 Step 5: Open the Application

Open a web browser and navigate to:

- http://localhost:5000

Within a few seconds, the connection status should display **Connected**.

13.1.4 Step 6: Register a Device

1. Open the Register page.
2. Enter a Device ID and PSK.
3. Click **Register Device**.
4. Select the device as the master device.

13.1.5 Step 7: Register a Second Device and Start Chat

1. Open another browser tab.
2. Register a second device using the same PSK.
3. Set it as the master device.
4. Open the Chat page in both tabs.
5. Send messages between the two devices.
6. The messages will be encrypted and decrypted automatically.

13.1.6 Step 8: Run Attack Simulations

1. Open the Attack page.
2. Select a device.
3. Choose an attack type (Replay, Spoof, MITM).
4. Run the simulation.
5. View the results in the output panel.
6. Monitor the dashboard for real-time logs.

Chapter 14

Results and Output

This chapter presents the outputs of the system along with screenshots of different modules. Each section highlights how the system behaves in real-time and demonstrates the implemented security features.

14.1 Device Registration and Master Selection

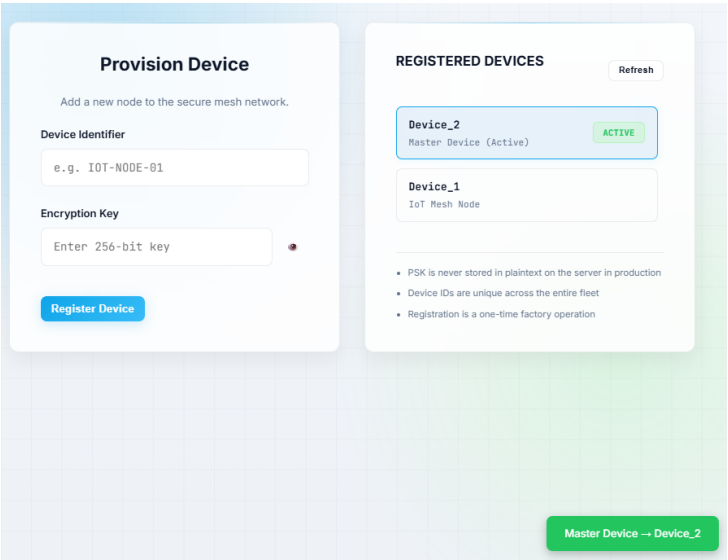


Figure 14.1: Device Registration and Master Device Selection

This interface allows users to register a new device by entering a device ID and a PSK. Once registered, the device appears in the available device list. The user can then select one device as the master device, which is used for further operations.

14.2 Chat Section (Device 1)

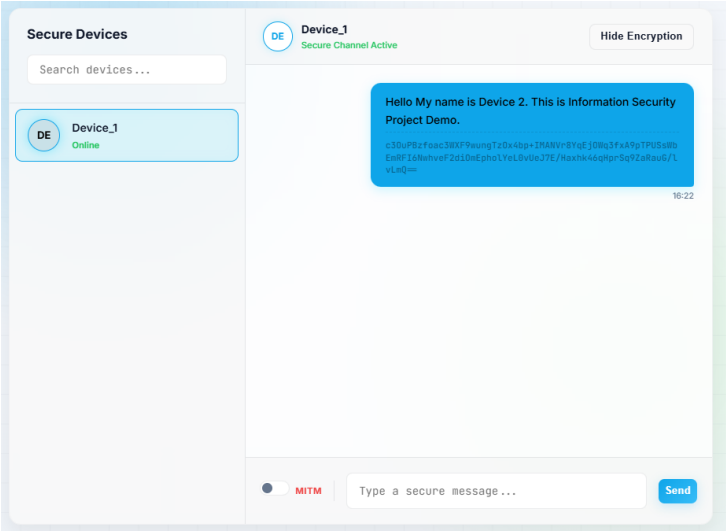


Figure 14.2: Secure Chat Interface from Device 1

This is the screenshot of the chat screen from the view point of Device 1. The messages are encrypted before sending to ensure secure communication. It also indicates the status of connectivity and active device.

14.3 Chat Section (Device 2)

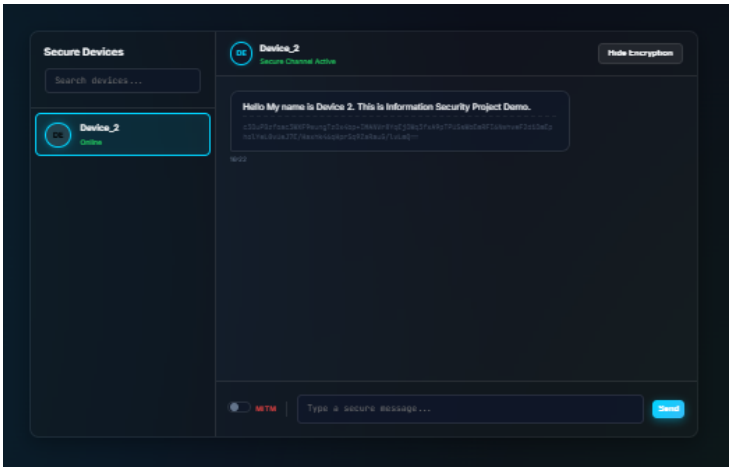


Figure 14.3: Secure Chat Interface from Device 2

This screen capture displays the conversation window from the perspective of Device 2. All received messages are automatically decrypted, thus confirming that the conversation among the two devices stays secure.

14.4 Attack Simulation

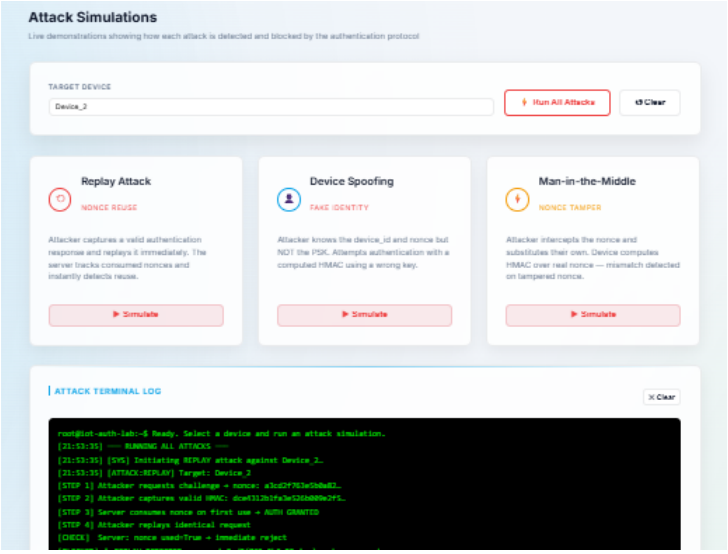


Figure 14.4: Attack Simulation Interface

This module illustrates the simulation process of attacks such as replay, spoofing, and MITM. It also explains how these attacks can be detected and prevented. This is made possible with the use of the output pane that clearly illustrates log messages for each process involved.

14.5 Cryptography Simulation and Avalanche Effect

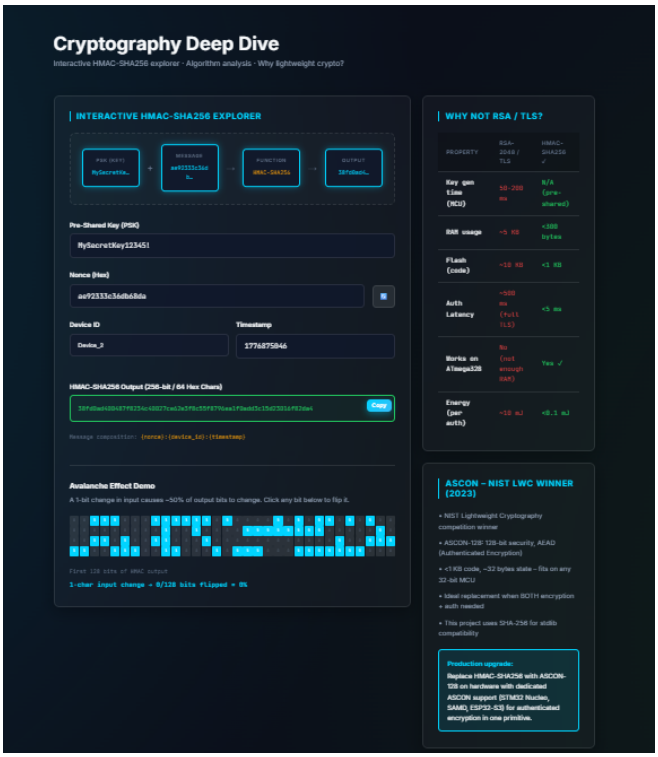


Figure 14.5: Cryptography Simulation and Avalanche Effect

In this module, we have seen examples of the usage of HMAC-SHA256 algorithm along with the avalanche effect. That means that a very slight difference in the input makes the output completely different.

14.6 Authentication Flow Simulation

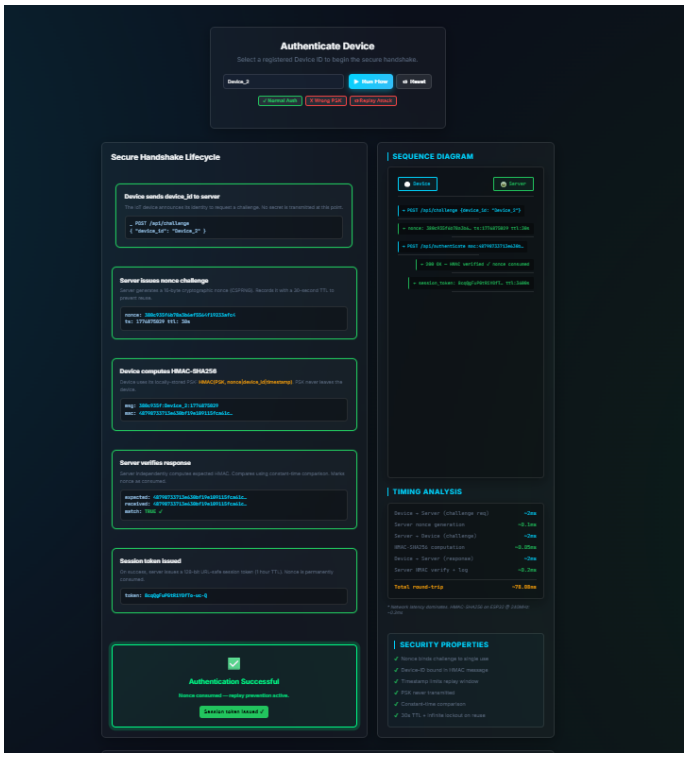


Figure 14.6: Authentication Flow Simulation

This part demonstrates the authentication procedure, which includes generation of nonces, calculating and verifying HMACs, and establishing the session. The chart explains the mechanism of establishing secure communications.

14.7 Backend Execution and Terminal Output

Listing 14.1: Backend Server Execution and Real-Time Logs

```
PS C:\Users\ravit\Downloads\Projects\Information_Security>
   python Backend/app.py Frontend

[DB]  SQLite initialised -> iot_security.db
[INFO] Serving frontend from: ...\Frontend

============================================================
  SIOTDA -- Real-Time WebSocket Edition
  Server  ->  http://localhost:5000
  Stop    ->  Ctrl+C
============================================================

[WS] Client connected
[JOIN] Socket joined room: Device_2

============================================================
```

```
[BACKEND RECEIVED]
  Sender   : Device_2
  Receiver : device_1
  MITM     : False
  IV       : N9IOVPubHE/tW39x
  Data     : c3OuPBzfoac3WXF9...
============================================================

[DELIVERED] Device_2 -> Device_1

============================================================
[BACKEND RECEIVED]
  Sender   : Device_2
  Receiver : Device_1
  MITM     : True
  IV       : dwPrah4ki1lMFtux
  Data     : TGJfaLDxxpqXxoQi...
============================================================
```

The backend's processes can be seen in the terminal output. Server startup, database initialization, WebSocket communication, and allocation of rooms take place here. Additionally, message encryption between devices and detection of MITM attacks is shown if simulations are turned on.

Log files show the process through which messages are sent, blocked, and received if there is any sort of suspicious behavior detected.

14.8 Limitations

- **Dependence on Shared PSK:** The solution relies on manually shared PSKs, which could prove difficult for widespread IoT scenarios.
- **Scalability Issues:** While good for small configurations, SQLite might have difficulties supporting large deployments.
- **MITM Attack Simulation:** The MITM attack is carried out at the application layer and thus cannot fully simulate actual attacks in the network layer.
- **Lack of Key Management Features:** Key rotation and secure key distribution functionalities are missing from the design.

Chapter 15

Conclusion

In this project, a secure and efficient authentication protocol for IoT devices has been developed by making use of lightweight cryptography techniques. In doing so, it has been shown that security can be effectively ensured in resource-constrained environments as long as appropriate algorithms such as HMAC-SHA256 and AES-256-GCM have been selected.

- Nonce based challenge response has been used for reliable device authentication.
- Replay, spoofing, MITM, and similar attacks have been successfully identified and countered.
- Real time communication was made possible by making use of WebSocket.
- AES-256-GCM cryptography has been used for ensuring data privacy and integrity.

Therefore, it has been demonstrated that a lightweight cryptographic solution provides a good tradeoff between performance and security.

Chapter 16

References

- H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, 1997.
- D. He, N. Kumar, J. Chen, and S. Lee, "Secure Authentication Scheme for IoT Applications," IEEE Internet of Things Journal, 2017.
- Research and Improvement of AES Encryption Algorithm Based on Three-Dimensional Chaotic Mapping, IEEE Xplore, 2026.